

UNIVERSITY OF LAY ADVENTISTS OF KIGALI

Faculty of Computing and Information Sciences

CYBER SECURITY INCIDENT MANAGEMENT SYSTEM

Final Report

Course: DPR400210 — Database Programming with Oracle Database

Academic Year 2025-2026

Student: MOUSSAVOU MAKAYA Sage Taliane | **ID:** 26587/2024

Instructor: Eric Maniraguha

Table of Contents

Table of Contents.....	2
1. Introduction & Problem Statement	3
1.1 Target Users	3
1.2 Objectives	3
2. Business Process Modeling (Phase II)	3
3. Logical Database Design (Phase III)	3
3.1 Normalization Proof.....	4
4. Database Creation (Phase IV).....	4
5. Table Implementation (Phase V)	4
6. PL/SQL Programming (Phase VI).....	4
7. Advanced Database Programming (Phase VII)	4
8. Conclusion	4

1. Introduction & Problem Statement

Organizations frequently track security incidents informally — through email threads or spreadsheets — with no consistent way to measure response time against severity, and no audit trail of who changed what. The Cyber Security Incident Management System (CSIMS) addresses this gap with a normalized Oracle database and a PL/SQL application layer that logs, tracks, and audits the full lifecycle of a security incident.

1.1 Target Users

- Employees (Reporters) — anyone who detects and reports a security event
- SOC Analysts — investigate, assign severity, and respond to incidents
- Security Managers — review and formally close incidents
- System Administrators — manage configuration data (severity levels, categories)

1.2 Objectives

- Centralize incident logging with structured, queryable data
- Enforce severity-based SLA response times
- Maintain a complete, tamper-evident status history
- Automatically audit every change to incident data
- Restrict changes to configuration data to controlled maintenance windows

2. Business Process Modeling (Phase II)

The incident lifecycle involves four actors operating across four swimlanes: Reporter, System (automated), SOC Analyst, and Security Manager. The full BPMN diagram with swimlanes is included as Appendix A.

End-to-end flow: Detect → Report → Log (OPEN) → Assign (IN_PROGRESS) → Investigate/Respond → Resolve (RESOLVED) → Review → Close (CLOSED). Every status transition is captured automatically by database triggers, independent of which actor performs the action — this is what makes the audit trail complete and reliable rather than dependent on manual logging discipline.

3. Logical Database Design (Phase III)

The schema consists of 10 entities normalized to Third Normal Form (3NF). The full Entity-Relationship Diagram is included as Appendix B. A summary of each entity follows:

Entity	Purpose	Key Relationships
DEPARTMENTS	Organizational units	Referenced by EMPLOYEES, ASSETS
EMPLOYEES	Reporters, analysts, managers, admins	FK to DEPARTMENTS
SEVERITY_LEVELS	LOW/MEDIUM/HIGH/CRITICAL + SLA hours	Referenced by INCIDENTS
INCIDENT_CATEGORIES	Phishing, Malware, DDoS, etc.	Referenced by INCIDENTS
ASSETS	Systems/devices that can be affected	FK to DEPARTMENTS
PUBLIC_HOLIDAYS	Reference dates for the business rule	Used by trigger logic
INCIDENTS	Core transactional record	FKs to 5 other tables
INCIDENT_STATUS_HISTORY	Full timeline of status changes	FK to INCIDENTS, EMPLOYEES
RESPONSE_ACTIONS	Actions taken to resolve an incident	FK to INCIDENTS, EMPLOYEES
AUDIT_LOG	Generic change log populated by triggers	Standalone, trigger-populated

3.1 Normalization Proof

1NF: every attribute holds a single atomic value; one-to-many facts (multiple status changes, multiple response actions per incident) are modeled as separate child tables rather than repeating columns.

2NF: every table uses a single-column surrogate primary key (IDENTITY-generated), so partial dependency on a composite key is structurally impossible.

3NF: no non-key attribute depends on another non-key attribute. For example, `response_time_hours` lives only in `SEVERITY_LEVELS` and is reached via JOIN — it is never duplicated into `INCIDENTS`, which would create a transitive dependency on `severity_id` rather than `incident_id`.

4. Database Creation (Phase IV)

The Oracle user/schema was created following the mandatory naming convention `StudentID_FirstName_CyberIncident_DB`, granted `CONNECT`, `RESOURCE`, and the privileges required to create views, sequences, triggers, and procedures.

```
CREATE USER Taliane_26587_2024;  
GRANT CONNECT, RESOURCE TO Taliane_26587_2024;  
GRANT CREATE VIEW, CREATE SEQUENCE, CREATE TRIGGER, CREATE PROCEDURE  
TO Taliane_26587_2024;
```

5. Table Implementation (Phase V)

All 10 tables were created with primary keys (IDENTITY columns), foreign keys enforcing referential integrity, NOT NULL constraints on mandatory fields, UNIQUE constraints where appropriate (e.g. employee email, department name), and CHECK constraints restricting enumerated values (employee role, incident status, severity name, asset type). Sample data covering four departments, six employees, four severity levels, five categories, four assets, five public holidays, and four sample incidents was inserted to support meaningful demo queries.

6. PL/SQL Programming (Phase VI)

Standalone PL/SQL objects:

`get_open_incidents_count` (function), `report_incident`, `assign_incident`, `resolve_incident` (procedures), and `list_overdue_incidents`.

Related operations were also grouped into a package, **`pkg_incident_mgmt`**, containing **`count_by_status`**, **`change_status`**, and **`print_incident_summary`**.

7. Advanced Database Programming (Phase VII)

Two row-level AFTER triggers on `INCIDENTS` provide automated auditing: `trg_incidents_audit` logs every INSERT/UPDATE/DELETE into `AUDIT_LOG`, and `trg_incidents_status_history` automatically records every status transition into `INCIDENT_STATUS_HISTORY`. The mandatory business rule (block INSERT/UPDATE/DELETE on weekdays and public holidays) is implemented as two compound triggers, `trg_protect_severity_levels` and `trg_protect_incident_categories`, applied to the reference/configuration tables rather than the operational `INCIDENTS` table — since incidents must be logged 24/7, but SLA and category definitions should only change during a controlled weekend maintenance window.

8. Conclusion

CSIMS demonstrates a complete, normalized Oracle database solving a realistic operational problem: centralizing incident tracking with enforced SLAs, automated auditing, and controlled configuration management. Every requirement in the course's Final Demonstration checklist — database structure, query execution, PL/SQL programs, triggers and packages, and the audit system — is implemented, tested, and documented, with the innovation component remaining to be finalized before submission.